Universidad Continental

Asignatura: **Estructura de Datos**

NRC: **30243**

Proyecto: **“Sistema de Gestión de Procesos”**

Integrantes: **Ore Buendia Jorch Noe**

INDICE

CAPÍTULO 1: Análisis del Problema

1. Descripción del problema
2. Requerimientos del sistema
   - Funcionales
   - No funcionales
3. Estructuras de datos propuestas
4. Justificación de la elección

Capítulo 2: Diseño de la Solución

1. Descripción de estructuras de datos y operaciones
2. Algoritmos principales
   - Pseudocódigo para agregar proceso
   - Pseudocódigo para cambiar estado del proceso
3. Diagramas de Flujo
4. Justificación del diseño

Capítulo 3: Solución Final

1. Código limpio, bien comentado y estructurado
2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación de datos
3. Manual de usuario

Capítulo 4: Evidencias de Trabajo en Equipo

1. Repositorio con Control de Versiones (Capturas de Pantalla)
   - Registro de commits claros y significativos que evidencien aportes individuales (proactividad)
   - Historial de ramas y fusiones si es aplicable
   - Evidencia por cada integrante del equipo
   - Enlace a la herramienta colaborativa
2. Plan de Trabajo y Roles Asignados

- Documento inicial donde se asignan tareas y responsabilidades
- Cronograma con fechas límite para cada entrega parcial
- Registro de reuniones o comunicación del equipo (Actas de reuniones)

CAPÍTULO 1: ANÁLISIS DEL PROBLEMA

1. Descripción del problema

En un sistema operativo se ejecutan varios procesos que necesitan ser organizados y administrados para utilizar correctamente los recursos disponibles. Cada proceso requiere tiempo de CPU y, en algunos casos, espacio en memoria para realizar sus tareas.

El problema surge cuando se necesita registrar, buscar, modificar y eliminar procesos, además de establecer un orden para su ejecución y controlar la memoria utilizada. Si estas operaciones no se gestionan adecuadamente, pueden presentarse dificultades en la organización y seguimiento de los procesos.

Por ello, este proyecto propone desarrollar un Sistema de Gestión de Procesos en C++, utilizando Dev-C++. El programa permitirá simular el registro y administración de procesos mediante una lista enlazada, organizar su ejecución mediante una cola de prioridad y representar la asignación y liberación de memoria mediante una pila.

De esta manera, se busca aplicar las estructuras de datos dinámicas estudiadas en clase para desarrollar un sistema funcional, organizado y fácil de utilizar.

2. Requerimientos del sistema

El Sistema de Gestión de Procesos debe cumplir con ciertos requerimientos que permitan administrar los procesos registrados, organizar su ejecución y controlar la memoria disponible. Estos requerimientos se dividen en funcionales y no funcionales.

1.2.1 Requerimientos funcionales

Son las funciones que debe ofrecer el sistema al usuario.

Código	Descripción
RF-01	Registrar procesos con su ID, nombre, prioridad y memoria requerida.
RF-02	Mostrar los procesos registrados y su información.
RF-03	Buscar un proceso por su ID.
RF-04	Eliminar procesos del registro.
RF-05	Modificar la prioridad o el estado de un proceso.
RF-06	Organizar la ejecución de procesos mediante una cola de prioridad.
RF-07	Administrar la memoria (asignar y liberar) utilizando una pila.
RF-08	Consultar los procesos en cola y los bloques de memoria.
RF-09	Guardar la información en un archivo.
RF-10	Recuperar los datos almacenados en el archivo.

1.2.2 Requerimientos no funcionales

Son las condiciones técnicas y de calidad que debe cumplir el sistema.

Código	Descripción
RNF-01	Desarrollo del sistema en C++ utilizando Dev-C++.
RNF-02	Uso de las estructuras de datos: lista enlazada, cola de prioridad y pila.
RNF-03	Interfaz por consola, sencilla y de fácil uso.
RNF-04	Validación de datos para evitar errores y registros duplicados.
RNF-05	Gestión de memoria dinámica y liberación de recursos.
RNF-06	Almacenamiento y recuperación de la información mediante archivos de texto.
RNF-07	Código modular y organizado en funciones.

3. Estructuras de datos propuestas

Para desarrollar el Sistema de Gestión de Procesos se utilizarán tres estructuras de datos dinámicas lineales: una lista enlazada, una cola de prioridad y una pila. Cada una permitirá realizar operaciones específicas dentro del programa.

Lista enlazada

Se utilizará para almacenar los procesos registrados. Cada nodo contendrá un identificador (ID), nombre, prioridad, estado, memoria requerida y un puntero que lo conecta con el siguiente nodo.

Mediante esta estructura se podrán registrar, buscar, modificar, eliminar y visualizar los procesos.

Cola de prioridad

Se utilizará para organizar los procesos que esperan ser ejecutados por la CPU. Los procesos se ordenarán según la prioridad asignada, permitiendo atender primero a los más importantes.

Esta estructura permitirá insertar procesos, consultar la cola y retirar el siguiente proceso para simular su ejecución.

Pila

Se utilizará para simular la asignación y liberación de bloques de memoria. Su funcionamiento seguirá el principio LIFO (último en entrar, primero en salir).

Permitirá asignar bloques, consultar la memoria utilizada y liberar el último bloque registrado.

4. Justificación de la elección

Las estructuras de datos seleccionadas son adecuadas porque permiten organizar las diferentes operaciones del Sistema de Gestión de Procesos de forma dinámica.

La **lista enlazada** facilita el registro y la eliminación de procesos sin establecer una cantidad fija de elementos, utilizando nodos conectados mediante punteros.

La **cola de prioridad** permite organizar los procesos según su importancia, simulando el orden en que son atendidos por la CPU.

Por su parte, la **pila** permite representar de manera sencilla la asignación y liberación de bloques de memoria, aplicando el principio LIFO.

La combinación de estas tres estructuras permite separar las funciones del sistema y aplicar los conocimientos adquiridos sobre nodos, punteros y memoria dinámica en C++.

Capítulo 2: Diseño de la Solución

1. Descripción de estructuras de datos y operaciones

El Sistema de Gestión de Procesos estará desarrollado mediante tres estructuras de datos dinámicas lineales: lista enlazada, cola de prioridad y pila. Estas estructuras permitirán administrar los procesos, organizar su ejecución y controlar la memoria utilizada durante la simulación.
Cada estructura tendrá sus propias operaciones y será implementada en C++ mediante nodos, punteros y funciones.

Lista enlazada (Gestor de Procesos)

Esta estructura permitirá almacenar los procesos registrados en el sistema. Cada nodo contendrá los datos del proceso y un puntero hacia el siguiente nodo.
Las operaciones principales serán:

- **Agregar proceso:** registra un nuevo proceso con su ID, nombre, prioridad y memoria requerida.
- **Buscar proceso:** localiza un proceso mediante su identificador.
- **Mostrar procesos:** presenta los procesos registrados y sus datos.
- **Eliminar proceso:** elimina un nodo de la lista y actualiza sus enlaces.
- **Modificar prioridad:** permite cambiar la prioridad asignada a un proceso.
- **Cambiar estado:** actualiza la situación de un proceso dentro del sistema.

Cola de prioridad (Planificador de CPU)

Se utilizará para organizar los procesos que esperan ser ejecutados según su nivel de prioridad. En el programa, los valores numéricos mayores representan una prioridad más alta.
Sus operaciones principales serán:

- **Encolar:** agrega un proceso a la cola según su prioridad.
- **Desencolar:** retira el primer proceso para simular su ejecución.
- **Mostrar cola:** permite visualizar el orden de atención.
- **Buscar en cola:** verifica si un proceso se encuentra esperando.

Pila (Gestor de Memoria)

Esta estructura permitirá simular la administración de bloques de memoria mediante el principio LIFO, donde el último bloque asignado será el primero en liberarse.
Las operaciones principales serán:

- **Asignar memoria:** incorpora un bloque al tope de la pila.
- **Liberar memoria:** retira el último bloque asignado.
- **Mostrar memoria:** presenta los bloques almacenados y la memoria utilizada.
- **Buscar bloque:** permite consultar los bloques asociados a un proceso.

El sistema también contará con funciones para guardar y recuperar información mediante archivos de texto, permitiendo conservar los datos entre diferentes ejecuciones del programa.

2. Algoritmos principales

En esta sección se presentan los algoritmos utilizados para registrar procesos y actualizar sus estados dentro del sistema. Estos permiten comprender cómo se realizan las operaciones antes de implementarlas en C++.

Pseudocódigo para agregar proceso

Este algoritmo permite registrar un proceso en la lista enlazada, verificando primero que los datos ingresados sean válidos.

```
ALGORITMO RegistrarProceso
INICIO
LEER id
SI id <= 0 O BuscarId(id) != NULO ENTONCES
MOSTRAR "ID invalido o repetido"
RETORNAR
FIN SI

LEER nombre
SI nombre = "" ENTONCES
MOSTRAR "Nombre invalido"
RETORNAR
FIN SI

LEER prioridad
SI prioridad < 1 ENTONCES
MOSTRAR "Prioridad invalida"
RETORNAR
FIN SI

LEER memoria
SI memoria <= 0 O memoria > 1024 ENTONCES
MOSTRAR "Memoria fuera del rango permitido"
RETORNAR
FIN SI

CREAR nuevo nodo
nuevo.id <- id
nuevo.nombre <- nombre
nuevo.prioridad <- prioridad
nuevo.memoriaRequerida <- memoria
nuevo.estado <- "REGISTRADO"
nuevo.siguiente <- lista
lista <- nuevo

MOSTRAR "Proceso registrado correctamente"
FIN
```

Pseudocódigo para cambiar estado del proceso

Este algoritmo representa cómo cambia el estado de un proceso cuando ingresa a la cola de prioridad y cuando termina su ejecución.

```
ALGORITMO CambiarEstado
INICIO
LEER id
proceso <- BuscarId(id)

SI proceso = NULO ENTONCES
MOSTRAR "Proceso no encontrado"
RETORNAR
FIN SI

MOSTRAR estado actual
MOSTRAR "1. REGISTRADO"
MOSTRAR "2. EN_COLA"
MOSTRAR "3. EJECUTADO"
LEER nuevoEstado

SI nuevoEstado < 1 O nuevoEstado > 3 ENTONCES
MOSTRAR "Estado invalido"
RETORNAR
FIN SI

SI nuevoEstado = 1 ENTONCES
RetirarDeCola(id)
proceso.estado <- "REGISTRADO"

SINO SI nuevoEstado = 2 ENTONCES
SI proceso.estado != "EN_COLA" ENTONCES
InsertarCola(id, proceso.prioridad)
FIN SI
proceso.estado <- "EN_COLA"

SINO
RetirarDeCola(id)
proceso.estado <- "EJECUTADO"
FIN SI
```

3. Diagramas de Flujo

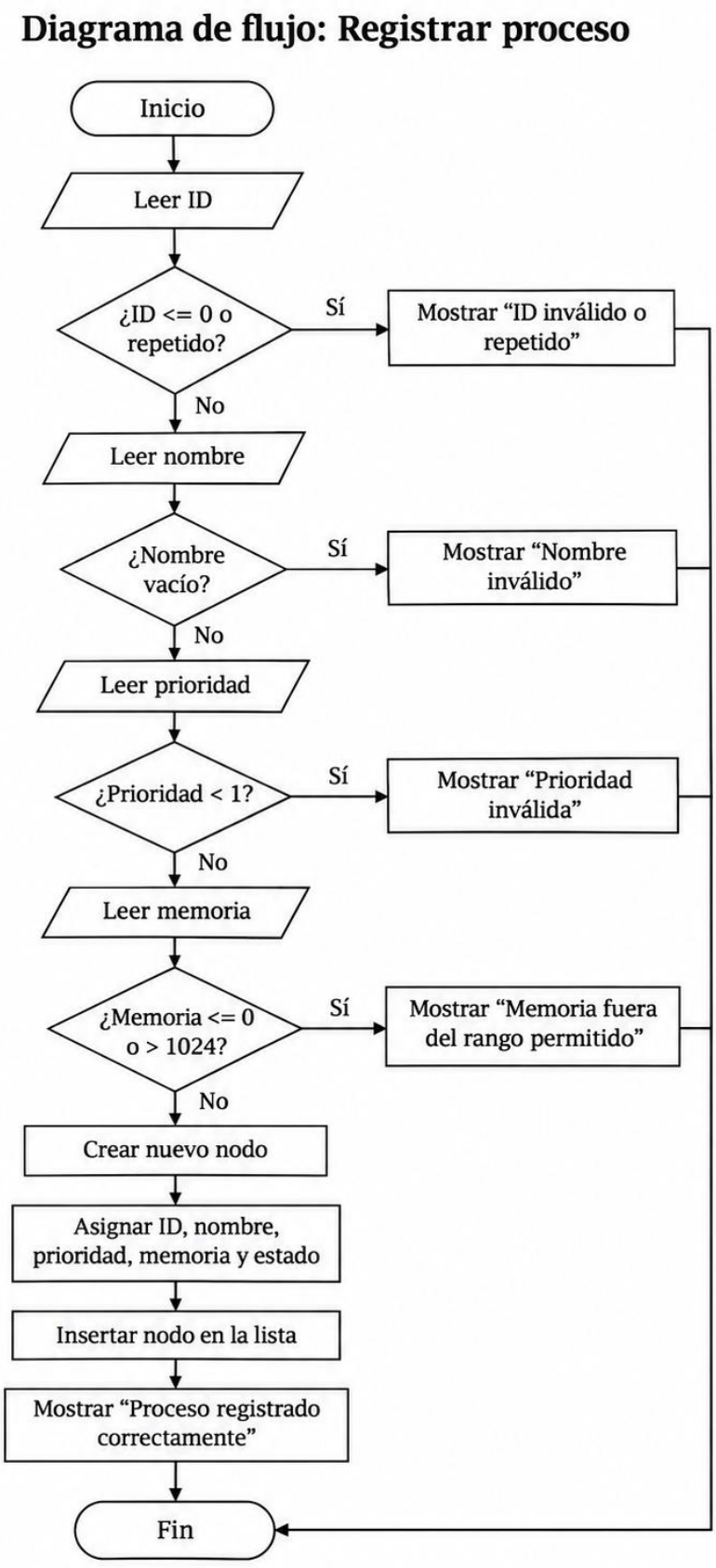
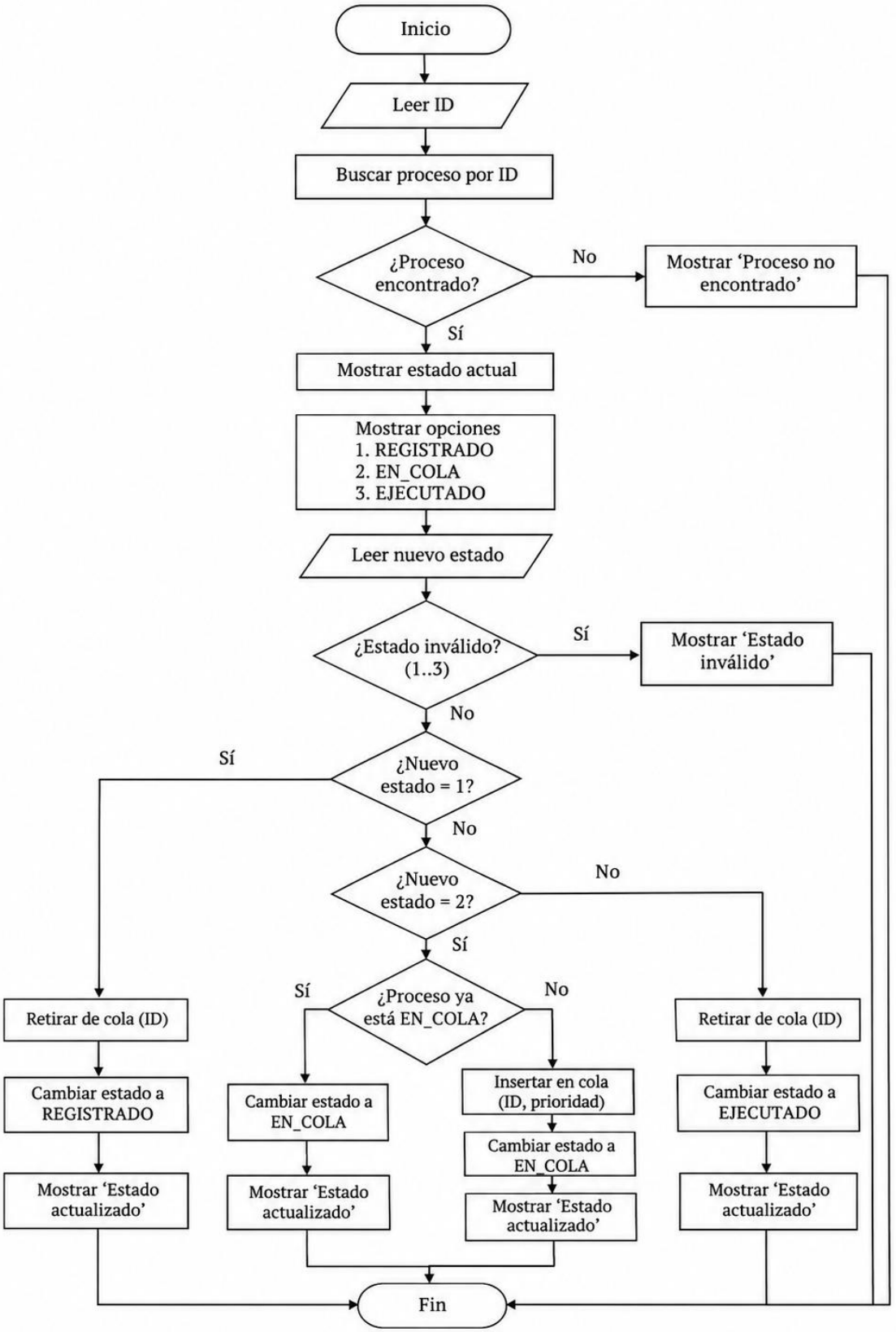


Diagrama de flujo: Cambiar estado del proceso



4. Justificación del diseño:

El diseño del Sistema de Gestión de Procesos se basa en el uso de tres estructuras de datos dinámicas lineales: lista enlazada, cola de prioridad y pila. Estas estructuras permiten organizar los procesos y administrar los recursos del sistema de manera sencilla y eficiente.

La **lista enlazada** permite registrar, buscar, modificar y eliminar procesos sin establecer una cantidad fija de elementos. Además, facilita la administración de los datos mediante nodos y punteros.

La **cola de prioridad** permite organizar los procesos según su importancia, de manera que aquellos con mayor prioridad sean atendidos primero. Esto ayuda a representar cómo se planifica la ejecución de procesos en la CPU.

Por otro lado, la **pila** permite simular la asignación y liberación de bloques de memoria mediante el principio LIFO, donde el último bloque ingresado es el primero en retirarse.

En cuanto a la eficiencia, las operaciones de inserción al inicio de la lista y de asignación o liberación en el tope de la pila pueden realizarse en tiempo constante **O(1)**. Las búsquedas y recorridos de las estructuras requieren un tiempo **O(n)**, mientras que insertar ordenadamente en la cola de prioridad también puede tomar **O(n)**.

Finalmente, el diseño permite organizar el código en funciones independientes, facilitando su comprensión y mantenimiento. De esta manera, se aplican los conocimientos adquiridos sobre estructuras de datos, nodos, punteros y memoria dinámica en C++.

CAPÍTULO 3: SOLUCIÓN FINAL

1. Código limpio, bien comentado y estructurado.

El Sistema de Gestión de Procesos fue desarrollado en C++ utilizando el entorno Dev-C++. Para su funcionamiento se implementaron tres estructuras de datos dinámicas lineales: una lista enlazada, una cola de prioridad y una pila.
El programa está organizado mediante funciones que permiten realizar las diferentes operaciones del sistema, facilitando la comprensión y el mantenimiento del código.
Gestor de procesos: utiliza una lista enlazada para registrar, buscar, modificar, eliminar y visualizar procesos. Cada proceso almacena su identificador, nombre, prioridad, memoria requerida y estado.

Planificador de CPU: emplea una cola de prioridad para organizar los procesos pendientes y simular su ejecución. Los procesos con mayor prioridad son atendidos primero.
Gestor de memoria: utiliza una pila para representar la asignación y liberación de bloques de memoria, siguiendo el principio LIFO. La simulación considera una capacidad máxima de 1024 MB.
Persistencia de datos: permite guardar y recuperar la información del programa mediante archivos de texto, evitando perder los registros al finalizar una sesión.
Además, se implementaron validaciones para evitar identificadores duplicados, datos incorrectos y operaciones sobre estructuras vacías.
A continuación, se presenta el código fuente completo del Sistema de Gestión de Procesos.

1. Bibliotecas utilizadas

Bibliotecas para entrada y salida, archivos, cadenas y límites de lectura.

```
#include <iostream>
#include <fstream>
#include <string>
#include <limits>
using namespace std;
```

2. Estructuras de datos,

variables globales y declaraciones de funciones Nodos, punteros globales y capacidad de memoria.

```
struct Proceso {
    int id;
    string nombre;
    int prioridad;
    int memoriaRequerida;
    string estado;
    Proceso* sig;
};
struct NodoCola {
    int id;
    int prioridad;
    NodoCola* sig;
};
struct Bloque {
    int idProceso;
    int cantidadMB;
    Bloque* sig;
};
```

```
Proceso* lista = NULL;
NodoCola* frente = NULL;
Bloque* cima = NULL;
```

```
const int MEMORIA_TOTAL = 1024;
```

3. Funciones auxiliares y lista enlazada (gestor de procesos) Leer, registrar, consultar, modificar, eliminar y mostrar procesos.

```
int leerEntero(const string& mensaje) {
    int n;
    while (true) {
        cout << mensaje;
        if (cin >> n) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return n;
        }
        if (cin.eof()) return 0;
        cout << "Entrada invalida. Ingrese un numero entero.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

Proceso* buscarId(int id) {
    for (Proceso* p = lista; p != NULL; p = p->sig)
        if (p->id == id) return p;
    return NULL;
}
void mostrarProceso(Proceso* p) {
    if (p == NULL) { cout << "Proceso no encontrado.\n"; return; }
    cout << "ID: " << p->id << " | Nombre: " << p->nombre
         << " | Prioridad: " << p->prioridad
         << " | Memoria requerida: " << p->memoriaRequerida << " MB"
         << " | Estado: " << p->estado << "\n";
}

void registrarProceso() {
    int id = leerEntero("ID del proceso: ");
    if (id <= 0 || buscarId(id) != NULL) {
        cout << "ID invalido o repetido.\n"; return;
    }
    string nombre;
    cout << "Nombre: "; getline(cin, nombre);
    if (nombre.empty()) { cout << "El nombre no puede estar vacio.\n"; return; }
    int prioridad = leerEntero("Prioridad (mayor numero = mayor prioridad): ");
    if (prioridad < 1) { cout << "La prioridad debe ser positiva.\n"; return; }
    int memoria = leerEntero("Memoria requerida (MB): ");
    if (memoria <= 0 || memoria > MEMORIA_TOTAL) {
        cout << "La memoria requerida debe estar entre 1 y 1024 MB.\n"; return;
    }
    Proceso* nuevo = new Proceso{id, nombre, prioridad, memoria, "REGISTRADO", lista};
    lista = nuevo;
    cout << "Proceso registrado correctamente.\n";
}

void eliminarProceso() {
    int id = leerEntero("ID a eliminar: ");
    Proceso* ant = NULL;
    Proceso* p = lista;
    while (p != NULL && p->id != id) { ant = p; p = p->sig; }
    if (p == NULL) { cout << "Proceso no encontrado.\n"; return; }
    if (tieneBloques(id)) {
        cout << "Libere primero los bloques de memoria asociados.\n"; return;
    }
    retirarDeCola(id);
    if (ant == NULL) lista = p->sig;
    else ant->sig = p->sig;
    delete p;
    cout << "Proceso eliminado correctamente.\n";
}

void buscarNombre() {
    string nombre;
    cout << "Nombre a buscar: "; getline(cin, nombre);
    bool encontrado = false;
```

```
    for (Proceso* p = lista; p != NULL; p = p->sig)
        if (p->nombre == nombre) { mostrarProceso(p); encontrado = true; }
    if (!encontrado) cout << "No se encontraron coincidencias.\n";
}

void modificarPrioridad() {
    int id = leerEntero("ID del proceso: ");
    Proceso* p = buscarId(id);
    if (p == NULL) { cout << "Proceso no encontrado.\n"; return; }
    int nueva = leerEntero("Nueva prioridad: ");
    if (nueva < 1) { cout << "La prioridad debe ser positiva.\n"; return; }
    p->prioridad = nueva;
    if (p->estado == "EN_COLA") {
        retirarDeCola(id);
        insertarCola(id, nueva);
    }
    cout << "Prioridad actualizada.\n";
}

void listarProcesos() {
    if (lista == NULL) { cout << "No hay procesos registrados.\n"; return; }
    for (Proceso* p = lista; p != NULL; p = p->sig) mostrarProceso(p);
}
```

4. Cola de prioridad (planificador de CPU) Agregar, retirar, ejecutar y buscar procesos en espera.

```
bool retirarDeCola(int id) {
    NodoCola* ant = NULL;
    NodoCola* act = frente;
    while (act != NULL && act->id != id) { ant = act; act = act->sig; }
    if (act == NULL) return false;
    if (ant == NULL) frente = act->sig;
    else ant->sig = act->sig;
    delete act;
    return true;
}

void insertarCola(int id, int prioridad) {
    NodoCola* n = new NodoCola{id, prioridad, NULL};
    if (frente == NULL || prioridad > frente->prioridad) {
        n->sig = frente; frente = n; return;
    }
    NodoCola* p = frente;
    while (p->sig != NULL && p->sig->prioridad >= prioridad) p = p->sig;
    n->sig = p->sig;
    p->sig = n;
}

void encolarProceso() {
    int id = leerEntero("ID para encolar: ");
    Proceso* p = buscarId(id);
    if (p == NULL) { cout << "Proceso no encontrado.\n"; return; }
    if (p->estado != "REGISTRADO") {
        cout << "Solo se pueden encolar procesos registrados y pendientes.\n"; return;
    }
    insertarCola(p->id, p->prioridad);
    p->estado = "EN_COLA";
    cout << "Proceso incorporado a la cola.\n";
}

// Permite modificar el estado desde el menu sin dejar una cola inconsistente.

void ejecutarProceso() {
    if (frente == NULL) { cout << "La cola esta vacia.\n"; return; }
    NodoCola* n = frente;
    frente = frente->sig;
    Proceso* p = buscarId(n->id);
    if (p != NULL) {
        p->estado = "EJECUTADO";
```

```
        cout << "Se ejecuto el proceso " << p->nombre << " (ID " << p->id << ").\n";
    }
    delete n;
}

void mostrarCola() {
    if (frente == NULL) { cout << "La cola esta vacia.\n"; return; }
    for (NodoCola* n = frente; n != NULL; n = n->sig)
        cout << "ID " << n->id << " | Prioridad " << n->prioridad << "\n";
}


// Busca un proceso que este esperando en la cola, sin cambiar su orden.

void buscarEnCola() {
    int id = leerEntero("ID a buscar en la cola: ");
    for (NodoCola* n = frente; n != NULL; n = n->sig) {
        if (n->id == id) {
            cout << "En cola: ID " << n->id << " | Prioridad " << n->prioridad << "\n";
            return;
        }
    }
    cout << "El proceso no esta en la cola.\n";
}

// Busca y suma los bloques asignados a un proceso dentro de la pila.
```

5. Pila (gestor de memoria)
Comprobar, asignar, liberar y consultar bloques de memoria.

```
bool tieneBloques(int id) {
    for (Bloque* b = cima; b != NULL; b = b->sig)
        if (b->idProceso == id) return true;
    return false;
}

void buscarEnPila() {
    int id = leerEntero("ID de proceso a consultar en la pila: ");
    int bloques = 0, totalMB = 0;
    for (Bloque* b = cima; b != NULL; b = b->sig) {
        if (b->idProceso == id) {
            bloques++;
            totalMB += b->cantidadMB;
        }
    }
    if (bloques == 0) cout << "No hay bloques asignados a ese ID.\n";
    else cout << "Bloques encontrados: " << bloques << " | Memoria: " << totalMB << " MB\n";
}

void asignarMemoria() {
    int id = leerEntero("ID del proceso: ");
    Proceso* p = buscarId(id);
    if (p == NULL) { cout << "Proceso no encontrado.\n"; return; }
    int mb = leerEntero("Cantidad de memoria (MB): ");
    if (mb <= 0) { cout << "La memoria debe ser positiva.\n"; return; }
    int ocupada = 0, usadaProceso = 0;
    for (Bloque* b = cima; b != NULL; b = b->sig) {
        ocupada += b->cantidadMB;
        if (b->idProceso == id) usadaProceso += b->cantidadMB;
    }
    if (mb > p->memoriaRequerida - usadaProceso) {
        cout << "Supera la memoria requerida por este proceso. Disponible para el proceso: "
             << p->memoriaRequerida - usadaProceso << " MB.\n";
        return;
    }
    if (mb > MEMORIA_TOTAL - ocupada) {
        cout << "Memoria insuficiente. Disponible: " << MEMORIA_TOTAL - ocupada << " MB.\n";
        return;
    }
    cima = new Bloque{id, mb, cima};
    cout << "Bloque asignado en la pila.\n";
}
```

```
void liberarMemoria() {
    if (cima == NULL) { cout << "La pila esta vacia.\n"; return; }
    Bloque* b = cima;
    cima = cima->sig;
    cout << "Bloque liberado: ID " << b->idProceso
         << " | " << b->cantidadMB << " MB.\n";
    delete b;
}

void mostrarMemoria() {
    if (cima == NULL) { cout << "La pila esta vacia.\n"; return; }
    cout << "De la cima hacia el fondo:\n";
    for (Bloque* b = cima; b != NULL; b = b->sig)
        cout << "Proceso " << b->idProceso << " | " << b->cantidadMB << " MB\n";
}
```

6. *Cambio de estado, persistencia y liberación de memoria Estados y datos guardados en archivo.*

```
void cambiarEstado() {
    int id = leerEntero("ID del proceso: ");
    Proceso* p = buscarId(id);
    if (p == NULL) { cout << "Proceso no encontrado.\n"; return; }
    cout << "Estado actual: " << p->estado << "\n";
    cout << "1. REGISTRADO (pendiente)\n";
    cout << "2. EN_COLA (en espera)\n";
    cout << "3. EJECUTADO (finalizado)\n";
    int nuevo = leerEntero("Nuevo estado: ");
    if (nuevo < 1 || nuevo > 3) {
        cout << "Estado invalido.\n"; return;
    }
    if (nuevo == 1) {
        retirarDeCola(id);
        p->estado = "REGISTRADO";
    } else if (nuevo == 2) {
        if (p->estado != "EN_COLA") insertarCola(p->id, p->prioridad);
        p->estado = "EN_COLA";
    } else {
        retirarDeCola(id);
        p->estado = "EJECUTADO";
    }
    cout << "Estado actualizado a " << p->estado << ".\n";
}

void limpiarDatos() {
    while (lista != NULL) {
        Proceso* p = lista; lista = lista->sig; delete p;
    }
    while (frente != NULL) {
        NodoCola* n = frente; frente = frente->sig; delete n;
    }
    while (cima != NULL) {
        Bloque* b = cima; cima = cima->sig; delete b;
    }
}

void guardarDatos() {
    ofstream out("procesos_datos.txt");
    if (!out) { cout << "No se pudo abrir el archivo.\n"; return; }
    out << "SGP2\n";
    int n = 0;
    for (Proceso* p = lista; p != NULL; p = p->sig) n++;
    out << n << '\n';
    for (Proceso* p = lista; p != NULL; p = p->sig)
        out << p->id << '\n' << p->nombre << '\n' << p->prioridad
            << '\n' << p->memoriaRequerida << '\n' << p->estado << '\n';
    n = 0;
    for (NodoCola* q = frente; q != NULL; q = q->sig) n++;
    out << n << '\n';
    for (NodoCola* q = frente; q != NULL; q = q->sig)
```

```
        out << q->id << '\n' << q->prioridad << '\n';
    n = 0;
    for (Bloque* b = cima; b != NULL; b = b->sig) n++;
    out << n << '\n';
    for (Bloque* b = cima; b != NULL; b = b->sig)
        out << b->idProceso << '\n' << b->cantidadMB << '\n';
    cout << "Datos guardados en procesos_datos.txt.\n";
}

void cargarDatos() {
    ifstream in("procesos_datos.txt");
    if (!in) { cout << "No existe el archivo para cargar.\n"; return; }
    string marca;
    getline(in, marca);
    if (marca != "SGP2") { cout << "Archivo antiguo o formato no reconocido; use datos guardados
con esta version (SGP2).\n"; return; }
    // Lee en estructuras auxiliares y cambia los datos solo si la lectura termina.
    Proceso* nuevaLista = NULL;
    Proceso* finLista = NULL;
    NodoCola* nuevaCola = NULL;
    NodoCola* finCola = NULL;
    Bloque* nuevaPila = NULL;
    Bloque* fondoPila = NULL;
    int cantidad;
    bool valido = bool(in >> cantidad) && cantidad >= 0 && cantidad <= 100000;
    if (valido) in.ignore(numeric_limits<streamsize>::max(), '\n');
    for (int i = 0; valido && i < cantidad; ++i) {
        int id, prioridad, memoria;
        string nombre, estado;
        valido = bool(in >> id);
        in.ignore(numeric_limits<streamsize>::max(), '\n');
        valido = valido && bool(getline(in, nombre)) && bool(in >> prioridad >> memoria);
        in.ignore(numeric_limits<streamsize>::max(), '\n');
        valido = valido && bool(getline(in, estado));
        valido = valido && id > 0 && prioridad > 0 && memoria > 0 && memoria <= MEMORIA_TOTAL &&
!nombre.empty()
            && (estado == "REGISTRADO" || estado == "EN_COLA" || estado == "EJECUTADO");
        if (valido) {
            Proceso* p = new Proceso{id, nombre, prioridad, memoria, estado, NULL};
            if (!nuevaLista) nuevaLista = p;
            else finLista->sig = p;
            finLista = p;
        }
    }
    int cantidadCola = 0;
    if (valido) valido = bool(in >> cantidadCola) && cantidadCola >= 0 && cantidadCola <=
100000;
    for (int i = 0; valido && i < cantidadCola; ++i) {
        int id, pr;
        valido = bool(in >> id >> pr) && id > 0 && pr > 0;
        if (valido) {
            NodoCola* q = new NodoCola{id, pr, NULL};
            if (!nuevaCola) nuevaCola = q;
            else finCola->sig = q;
            finCola = q;
        }
    }
    int cantidadBloques = 0;
    if (valido) valido = bool(in >> cantidadBloques) && cantidadBloques >= 0 && cantidadBloques
<= 100000;
    for (int i = 0; valido && i < cantidadBloques; ++i) {
        int id, mb;
        valido = bool(in >> id >> mb) && id > 0 && mb > 0;
        if (valido) {
            Bloque* b = new Bloque{id, mb, NULL};
            if (!nuevaPila) nuevaPila = b;
            else fondoPila->sig = b;
            fondoPila = b;
        }
    }
    // Validaciones de consistencia entre lista, cola y pila antes de sustituir datos.
    if (valido) {
        for (Proceso* p = nuevaLista; p != NULL; p = p->sig) {
            for (Proceso* otro = p->sig; otro != NULL; otro = otro->sig)
```

```
                if (p->id == otro->id) valido = false;
            int vecesCola = 0;
            for (NodoCola* q = nuevaCola; q != NULL; q = q->sig)
                if (q->id == p->id) vecesCola++;
            if ((p->estado == "EN_COLA" && vecesCola != 1) ||
                (p->estado != "EN_COLA" && vecesCola != 0)) valido = false;
        }
        for (NodoCola* q = nuevaCola; q != NULL; q = q->sig) {
            Proceso* relacionado = nuevaLista;
            while (relacionado && relacionado->id != q->id) relacionado = relacionado->sig;
            if (!relacionado || relacionado->prioridad != q->prioridad ||
                (q->sig && q->prioridad < q->sig->prioridad)) valido = false;
        }
        int ocupada = 0;
        for (Bloque* b = nuevaPila; b != NULL; b = b->sig) {
            ocupada += b->cantidadMB;
            bool existe = false;
            for (Proceso* p = nuevaLista; p != NULL; p = p->sig)
                if (p->id == b->idProceso) existe = true;
            if (!existe || ocupada > MEMORIA_TOTAL) valido = false;
        }
        for (Proceso* p = nuevaLista; p != NULL; p = p->sig) {
            int usada = 0;
            for (Bloque* b = nuevaPila; b != NULL; b = b->sig)
                if (b->idProceso == p->id) usada += b->cantidadMB;
            if (usada > p->memoriaRequerida) valido = false;
        }
    }
    if (!valido) {
        while (nuevaLista) { Proceso* p = nuevaLista; nuevaLista = p->sig; delete p; }
        while (nuevaCola) { NodoCola* q = nuevaCola; nuevaCola = q->sig; delete q; }
        while (nuevaPila) { Bloque* b = nuevaPila; nuevaPila = b->sig; delete b; }
        cout << "Archivo incompleto o con datos invalidos.\n";
        return;
    }
    limpiarDatos();
    lista = nuevaLista;
    frente = nuevaCola;
    cima = nuevaPila;
    cout << "Datos recuperados correctamente.\n";
}
```

7. Menú principal y función main

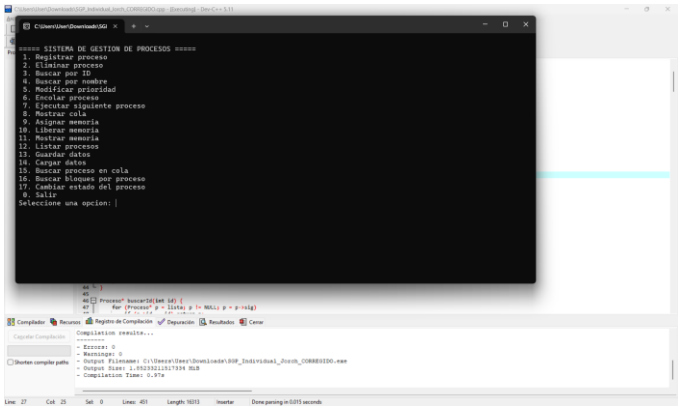
Menú de 17 opciones y salida.

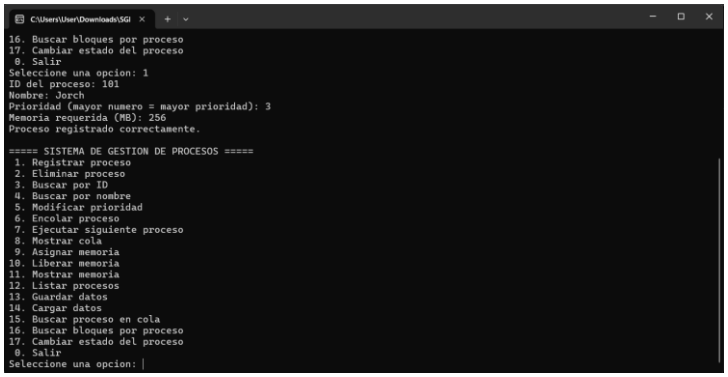
```
void mostrarMenu() {
    cout << "\n===== SISTEMA DE GESTION DE PROCESOS =====\n"
         << " 1. Registrar proceso\n 2. Eliminar proceso\n"
         << " 3. Buscar por ID\n 4. Buscar por nombre\n"
         << " 5. Modificar prioridad\n 6. Encolar proceso\n"
         << " 7. Ejecutar siguiente proceso\n 8. Mostrar cola\n"
         << " 9. Asignar memoria\n10. Liberar memoria\n"
         << "11. Mostrar memoria\n12. Listar procesos\n"
         << "13. Guardar datos\n14. Cargar datos\n"
         << "15. Buscar proceso en cola\n16. Buscar bloques por proceso\n"
         << "17. Cambiar estado del proceso\n 0. Salir\n";
}

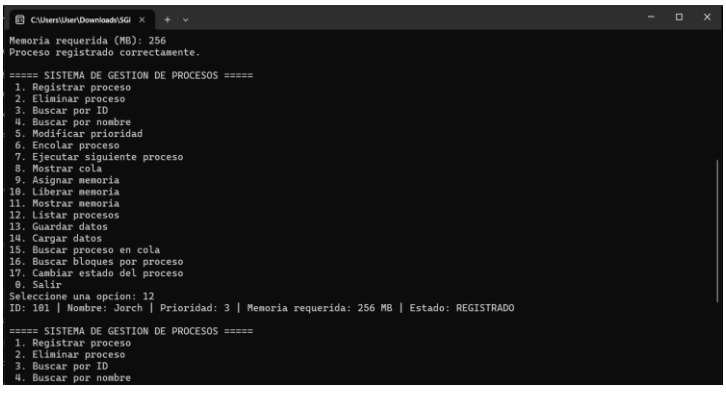
int main() {
    int opcion;
    do {
        mostrarMenu();
        opcion = leerEntero("Seleccione una opcion: ");
        switch (opcion) {
            case 1: registrarProceso(); break;
            case 2: eliminarProceso(); break;
            case 3: mostrarProceso(buscarId(leerEntero("ID a buscar: "))); break;
            case 4: buscarNombre(); break;
            case 5: modificarPrioridad(); break;
            case 6: encolarProceso(); break;
            case 7: ejecutarProceso(); break;
```

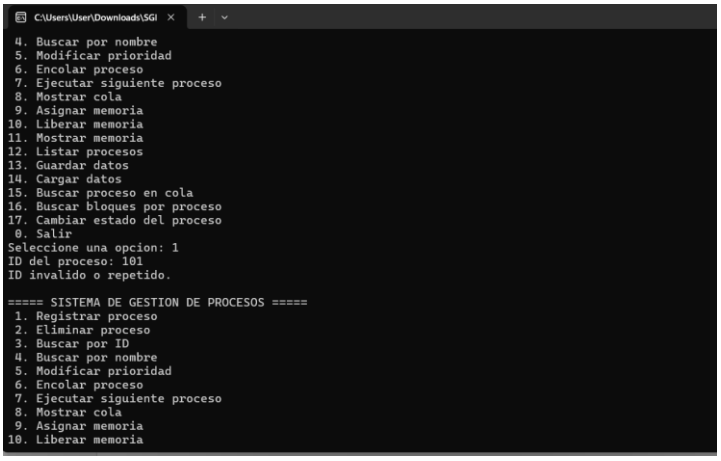
```
            case 8: mostrarCola(); break;
            case 9: asignarMemoria(); break;
            case 10: liberarMemoria(); break;
            case 11: mostrarMemoria(); break;
            case 12: listarProcesos(); break;
            case 13: guardarDatos(); break;
            case 14: cargarDatos(); break;
            case 15: buscarEnCola(); break;
            case 16: buscarEnPila(); break;
            case 17: cambiarEstado(); break;
            case 0: cout << "Saliendo del programa.\n"; break;
            default: cout << "Opcion no reconocida.\n";
        }
    } while (opcion != 0);
    limpiarDatos();
    return 0;
}
```

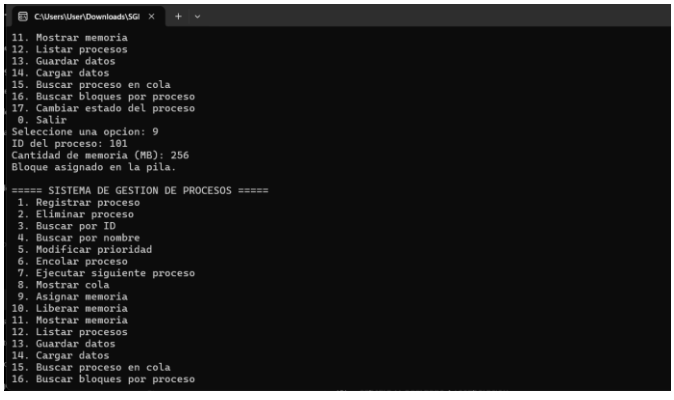
2. *Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación de datos*

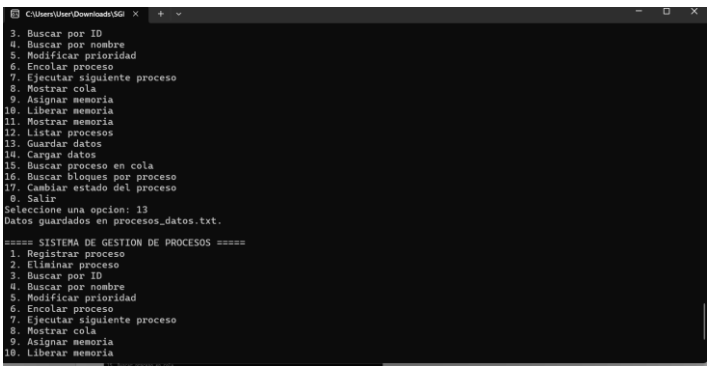












3. Manual de usuario

1 Introducción

El Sistema de Gestión de Procesos es una aplicación desarrollada en C++ que simula la administración de procesos de un sistema operativo mediante listas enlazadas, colas de prioridad y pilas.
Cuenta con un menú interactivo que permite registrar procesos, gestionar su ejecución, administrar memoria y guardar o recuperar información.

2 Requisitos del sistema

Para ejecutar el programa se necesita una computadora con Windows y un entorno compatible con C++, como Dev-C++. El sistema funciona mediante consola y no requiere conexión a Internet.

3 Instrucciones de ejecución

1. *Abrir Dev-C++ y cargar el archivo SGP_Individual_Jorch_CORREGIDO.cpp.*
2. *Presionar **F11** para compilar y ejecutar.*
3. *Esperar a que aparezca el menú principal.*
4. *Ingresar el número de la opción deseada y presionar **Enter**.*
5. *Seguir las indicaciones mostradas en pantalla.*

4 Descripción de las opciones del menú

Opción	***Función***	***Descripción***
1	*Registrar proceso*	*Registra un proceso con ID, nombre, prioridad y memoria requerida.*
2	*Eliminar proceso*	*Elimina un proceso mediante su ID.*
3	*Buscar por ID*	*Consulta un proceso por su identificador.*
4	*Buscar por nombre*	*Localiza un proceso mediante su nombre.*
5	*Modificar prioridad*	*Actualiza la prioridad de un proceso.*
6	*Encolar proceso*	*Agrega un proceso a la cola de ejecución.*
7	*Ejecutar siguiente proceso*	*Simula la ejecución según la prioridad.*
8	*Mostrar cola*	*Visualiza los procesos pendientes de ejecución.*
9	*Asignar memoria*	*Asigna memoria a un proceso registrado.*
10	*Liberar memoria*	*Libera memoria siguiendo el principio LIFO.*
11	*Mostrar memoria*	*Muestra los bloques de memoria asignados.*
12	*Listar procesos*	*Presenta los procesos y sus datos registrados.*
13	*Guardar datos*	*Almacena la información en un archivo.*
14	*Cargar datos*	*Recupera los datos guardados.*
15	*Buscar proceso en cola*	*Comprueba si un proceso está en la cola.*
16	*Buscar bloques por proceso*	*Consulta los bloques asociados a un proceso.*
17	*Cambiar estado del proceso*	*Cambia el estado a REGISTRADO, EN_COLA o EJECUTADO.*
0	*Salir*	*Finaliza el programa.*

5 Ejemplo de uso

Para registrar y consultar un proceso:

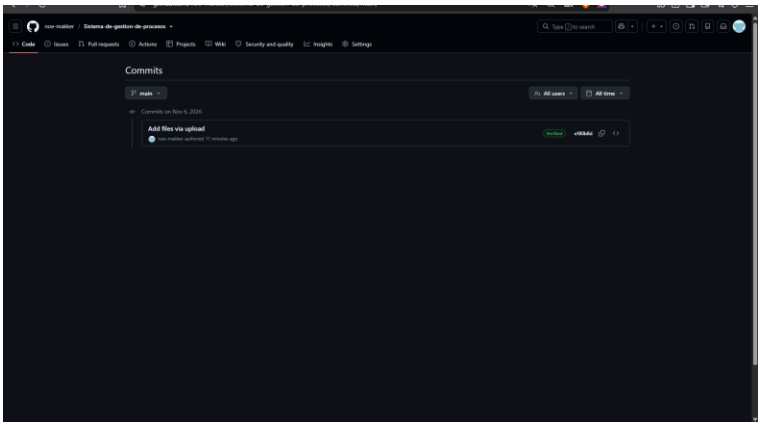
1. *Seleccionar la opción **1. Registrar proceso**.*
2. *Ingresar los siguientes datos:*
    - *ID: 101*
    - *Nombre: editor*
    - *Prioridad: 3*
    - *Memoria requerida: 256 MB*
3. *Seleccionar la opción **12** para comprobar que el proceso fue registrado.*
4. *Seleccionar la opción **6** para agregarlo a la cola.*
5. *Utilizar la opción **8** para visualizarlo en espera de ejecución.*

6 Recomendaciones de uso

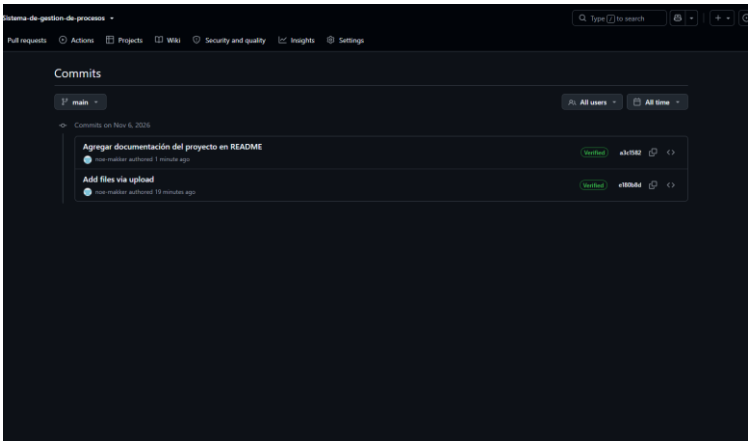
Se recomienda ingresar datos válidos, evitar identificadores duplicados y comprobar la existencia de los procesos antes de modificarlos o eliminarlos.
La memoria requerida debe ser mayor que cero y no superar los 1024 MB establecidos para la simulación. También se recomienda guardar los datos antes de cerrar el programa.
7 Cierre del programa
*Para finalizar, seleccionar la opción **0. Salir** desde el menú principal.*

Capítulo 4: Evidencias de Trabajo en Equipo

1. Registro de commits claros y significativos que evidencien aportes individuales (proactividad).



La captura muestra el primer registro de cambios del proyecto, correspondiente a la incorporación del código fuente del Sistema de Gestión de Procesos en la rama principal.



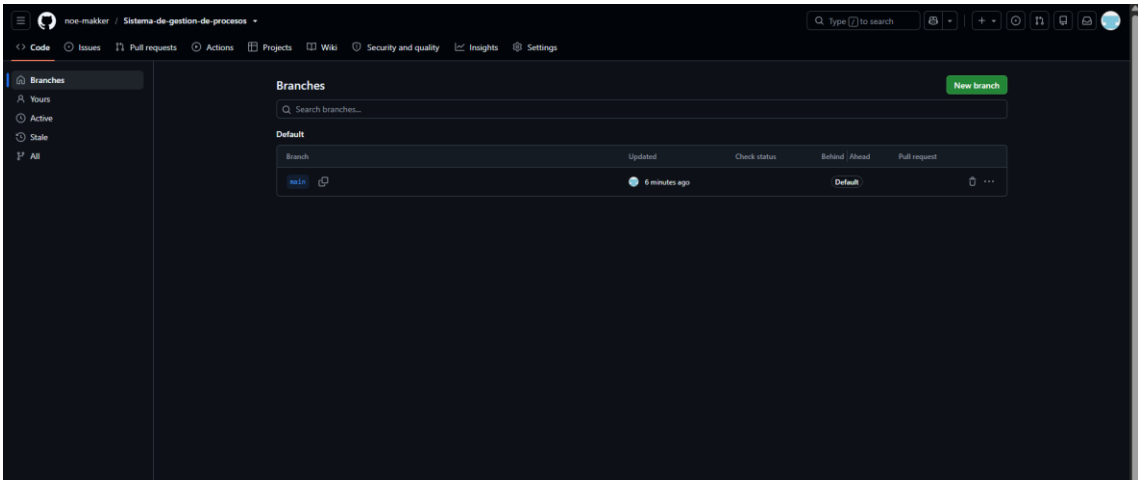
Registro de cambios del proyecto en GitHub.

Se presenta el historial de commits correspondiente a la incorporación del código fuente y la documentación del Sistema de Gestión de Procesos.

2. *Historial de ramas y fusiones si es aplicable.*

El desarrollo del Sistema de Gestión de Procesos se realizó utilizando la rama principal (main) del repositorio de GitHub.

Debido a que el proyecto fue desarrollado individualmente, no fue necesario crear ramas adicionales ni realizar fusiones. Los cambios se registraron directamente en la rama principal mediante commits.



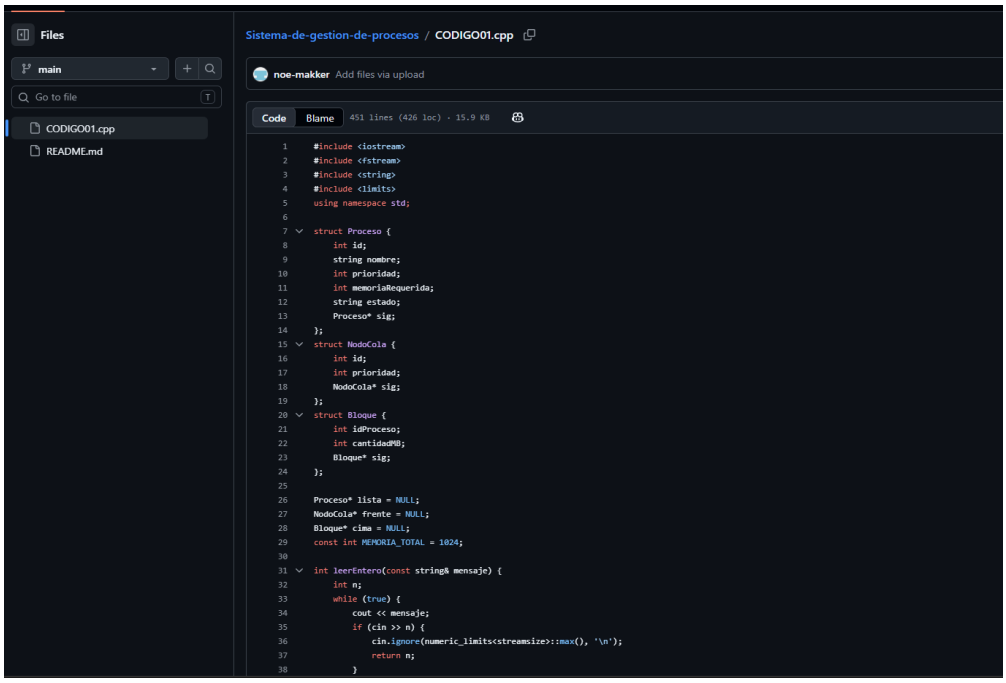
3. *Evidencia por cada integrante del equipo.*

El desarrollo del Sistema de Gestión de Procesos estuvo a cargo de un solo estudiante, quien asumió las responsabilidades de análisis, diseño, programación, pruebas y documentación.

Durante el proyecto se realizaron las siguientes actividades:

- Análisis de los requerimientos del sistema.
- Diseño de algoritmos y diagramas de flujo.
- Implementación de la lista enlazada, cola de prioridad y pila en C++.
- Desarrollo de las funciones de registro, búsqueda, modificación y eliminación de procesos.
- Implementación de la gestión de memoria y persistencia de datos.
- Compilación, pruebas y elaboración del informe técnico.

Como evidencia del trabajo realizado, se presenta el código fuente CODIGO01.cpp, almacenado en el repositorio de GitHub, junto con su historial de cambios.



Código fuente desarrollado en C++ y almacenado en GitHub.

4. Enlace a la herramienta colaborativa

Enlace a la herramienta colaborativa

El código fuente y la documentación del proyecto se encuentran almacenados en GitHub.

Enlace del repositorio: https://github.com/noe-makker/Sistema-de-gestion-de-procesos

2. Plan de Trabajo y Roles Asignados

1 .Documento inicial de tareas y responsabilidades

2.1 Documento inicial de tareas y responsabilidades

Para el desarrollo de la versión individual del Sistema de Gestión de Procesos, asumí la responsabilidad de realizar las actividades de análisis, diseño, programación, pruebas y documentación.

Responsable del proyecto: Jorch Noé Oré Buendía

Rol	Responsable	Actividades realizadas
Analista	Jorch Noé	Identificación del problema y definición de los requerimientos del sistema.
Diseñador	Jorch Noé	Elaboración de pseudocódigos, diagramas de flujo y selección de estructuras de datos.

Programador	Jorch Noé	Implementación del sistema en C++ utilizando listas enlazadas, colas de prioridad y pilas.
Responsable de pruebas	Jorch Noé	Compilación del programa y realización de pruebas de funcionamiento.
Documentador	Jorch Noé	Elaboración del informe técnico, manual de usuario y organización de evidencias en GitHub.

Esta distribución refleja que las responsabilidades de la versión individual del proyecto estuvieron a mi cargo, con el propósito de demostrar los conocimientos adquiridos durante el curso.

2 . Cronograma de actividades y fechas límite

Etapa	Actividad	Fecha límite	Estado
1	Análisis del problema y definición de requerimientos.	06/11/2026	**Completado**
2	Diseño de estructuras de datos, pseudocódigos y diagramas de flujo.	06/11/2026	**Completado**
3	Implementación del programa en C++.	06/11/2026	**Completado**
4	Compilación y pruebas de funcionamiento.	06/11/2026	**Completado**
5	Elaboración del informe y manual de usuario.	06/11/2026	**Completado**
6	Creación del repositorio de GitHub y registro de cambios.	06/11/2026	**Completado**
7	Revisión final y entrega del proyecto.	06/11/2026	**En proceso**